

HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

# Module 10

## Agent Prism for Monitoring, Observability and Enterprise Control

*The monitoring lens from Module 01, now hands-on: traces, failure patterns, token/cost leakage, quality signals and adoption — made visible, not assumed.*



DURATION

2 Hours · Module 10 of 12



FORMAT

Guided Ingest + Compare-Runs Lab



DELIVERED FOR

Honeywell Embedded Engineering

# How We'll Spend These Two Hours

Six connected moves — from why monitoring matters to telemetry tied back to engineering KPIs.



01

## Recall & Why Monitoring Matters

15 min

Agent Prism as the monitoring lens from Module 01 — now put into practice.



02

## Traces, Replay & Drift

20 min

Stepping back through what an agent actually did, and whether output is drifting.



03

## Failure Patterns & Cost Leakage

20 min

Common ways agent runs go wrong, and where token spend quietly disappears.



04

## Quality Signals, Adoption & Thresholds

15 min

Turning raw telemetry into scorecards, alerts, and an operational review rhythm.



05

## Ingest, Inspect, Compare

40 min

Load real run telemetry, inspect failures, and compare successful vs. poor-quality runs.



06

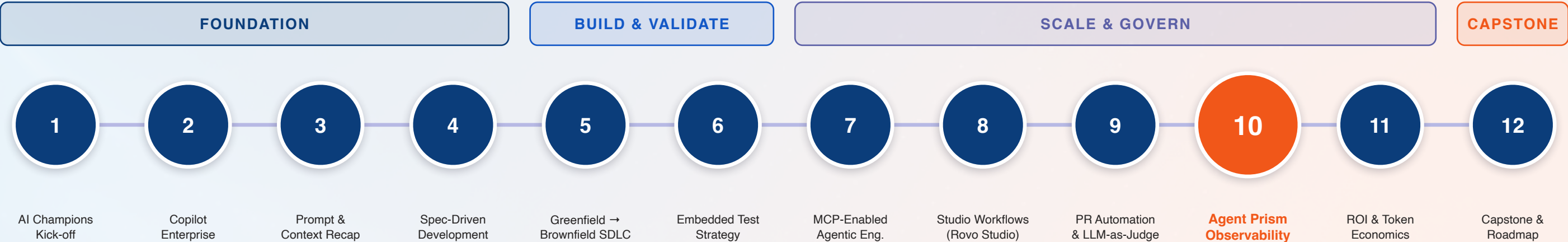
## Connect to Engineering KPIs

10 min

Tie today's findings back to Module 06's test evidence and Module 09's PR quality gates.

# Module 10 in the 12-Module Journey

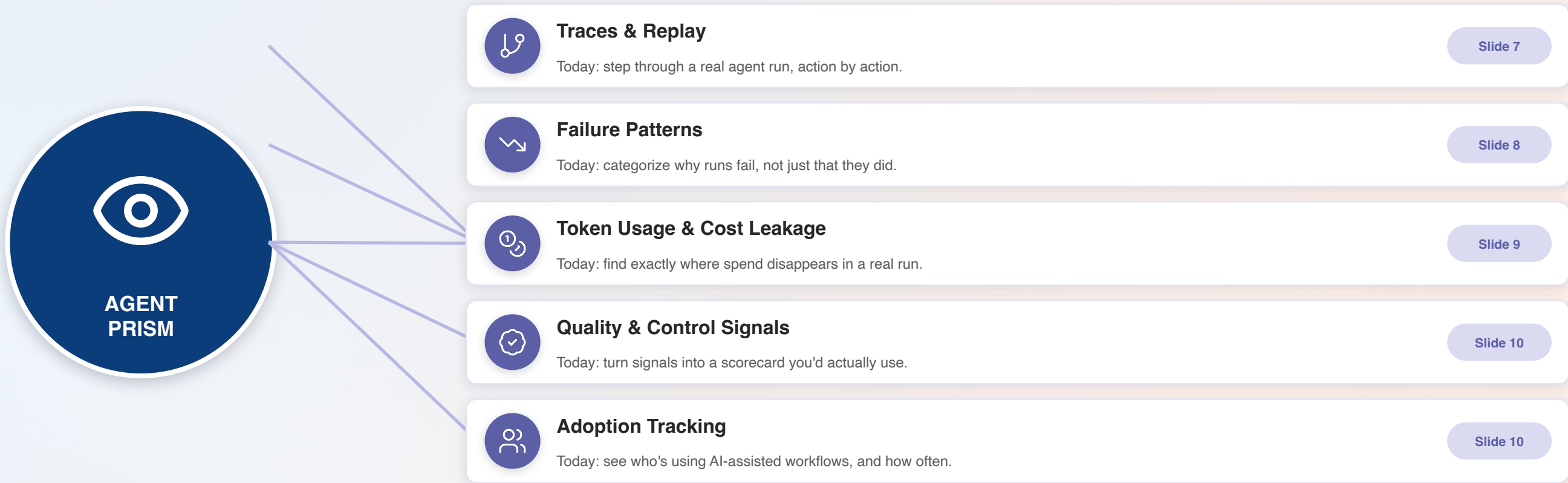
The concept from Module 01 becomes hands-on practice — right before it feeds the ROI model in Module 11.



**Why this matters:** Every module since Module 01 has generated something Agent Prism can observe — Copilot sessions, MCP workflows, PR quality gates. Module 11's token economics and ROI model are built entirely on what gets measured here.

# Recall: Agent Prism, From Concept to Practice

Module 01 introduced this as the monitoring lens. Today, every spoke gets a hands-on exercise.

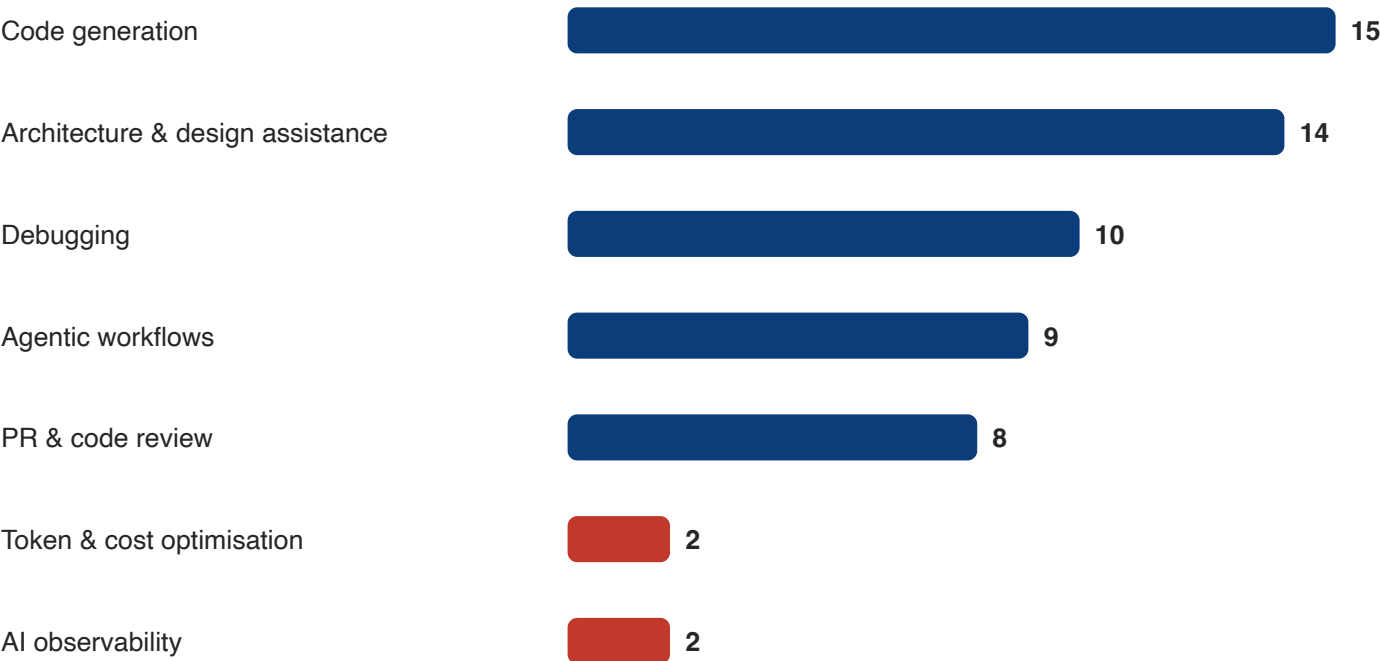




# The Blind Spot This Cohort Didn't Ask For

*Honest calibration data — the least-requested capability is arguably the one that makes every other one provable.*

## Which AI Capabilities Would Help Your Team Most?



(Full ranked list of 14 capabilities — top 5 and bottom 2 shown)



## Tied for last: 2 of 19

AI observability and token/cost optimisation ranked dead last, tied with build & CI assistance — the same 2 of 19 who ranked everything else highly.

That's not surprising — nobody asks to monitor a tool before they trust it enough to use it daily. But 16 of 19 already use AI daily or regularly (Module 02). The trust is here. The visibility isn't yet.

# Why Monitoring Agentic Engineering Matters

*Every earlier module produces agent activity — without this one, most of it is invisible.*

## WITHOUT OBSERVABILITY

- An agent failure is invisible until someone notices bad output downstream.
- Token spend accumulates with no one able to say where it went, or why.
- “Is adoption actually working?” gets answered with anecdotes, not data.
- A quietly drifting agent keeps running the same way until it causes real damage.
- Governance conversations happen with no evidence to ground them in.



## WITH AGENT PRISM

- A failed run is traceable to the exact step where it went wrong.
- Token spend is attributable to specific workflows — leakage gets named, not guessed at.
- Adoption tracking gives a real number: who's using what, and how often.
- Drift shows up as a trend before it becomes an incident.
- Governance & ROI conversations in Module 11 start from evidence, not opinion.

# Observability Concepts: Traces, Replay & Drift

*Three ways of asking the same question: what actually happened, and is it changing?*



## Traces

The full step-by-step record

Every tool call, decision & context pull an agent made, in order — the raw material everything else is built from.



## Replay

Step back through a run, exactly

Re-run a trace step by step to see precisely where a good or bad outcome was decided — not just what the final output was.



## Prompt / Output Drift

Change you'd otherwise miss

The same task type producing steadily different results over weeks — invisible run by run, obvious over a trend line.

# Failure Patterns: What Goes Wrong, and How to Spot It

*Six categories — naming the pattern is most of the work of fixing it.*



## Tool Call Failure

A build, test or API call fails mid-workflow — visible in the trace as a hard stop.



## Retry Loops

The agent repeats the same failing action without adapting — a clear trace signature.



## Context Misses

The agent acts without context it should have had — visible as an unexplained wrong turn.



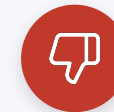
## Output Drift

Quality degrades gradually across similar tasks — only visible in aggregate, not per-run.



## Scope Creep

An agent modifies more than the task required — the Module 07 boundary being tested.



## Silent Low Confidence

A run completes but should have escalated — the Module 09 judge miscalibrated.

**Today's lab exercise:** you'll be given real run telemetry and asked to identify which of these six patterns explains what happened — before you're told the answer.



# Token Usage & Cost-Leak Visibility

*Module 03 taught you to spend tokens deliberately. This is where you find out if it worked.*



## Where Spend Is Expected

Task framing, scoped code context, targeted test generation — the Module 03 discipline, working as intended.

Normal



## Where Spend Gets Suspicious

Repeated re-explanation of the same context, oversized repository dumps, retry loops re-sending full history.

Investigate



## Where Leakage Actually Hides

Context inflation nobody scoped, low-confidence runs re-tried without a fix, workflows nobody remembers enabling.

Leak

**The tell:** leakage rarely looks dramatic in a single run — it shows up as a slow upward trend across many runs of the same workflow.

# Quality & Control Signals, and Adoption Tracking

*Two different questions — “is it good?” and “is anyone using it?” — both worth tracking on purpose.*

## QUALITY & CONTROL SIGNALS



Module 09's LLM-as-Judge scores, aggregated over time by workflow type



Regression rate on brownfield changes — Module 06's discipline, measured



Escalation rate — trending down could mean improving, or miscalibrating

## ADOPTION TRACKING



Who's actually running MCP workflows & studio automations, and how often



Adoption by GBE — PA / IA / BA — not just an aggregate programme number

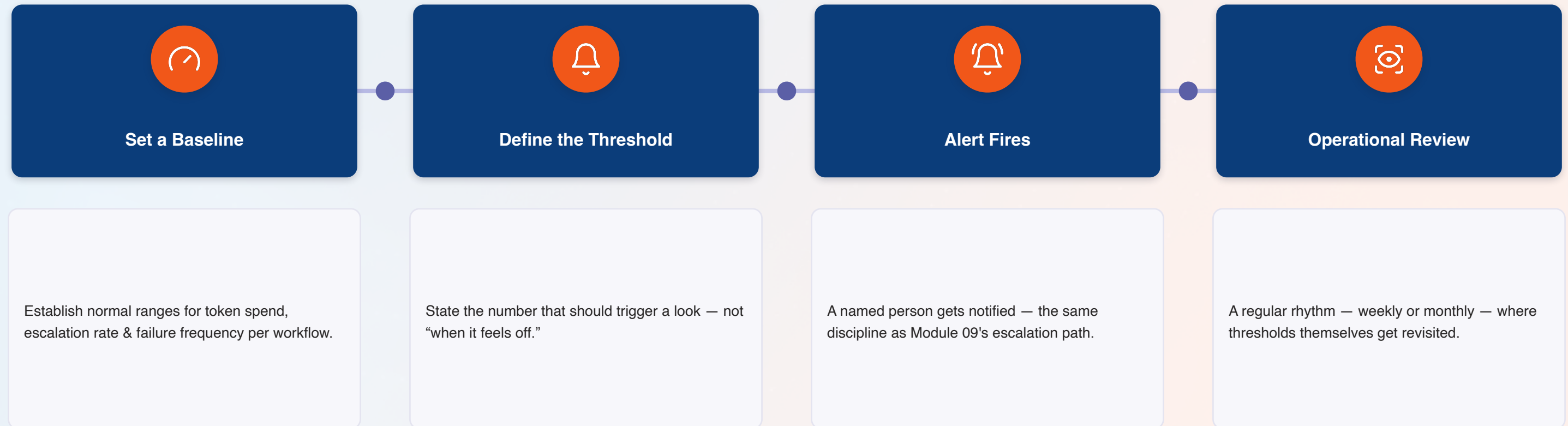


Trend over the 30-day catch-up window from Module 01's free value-adds

**Neither signal alone is enough:** high adoption with falling quality is a warning, not a win — and vice versa.

# Thresholds & Alerts: Turning Signals Into Action

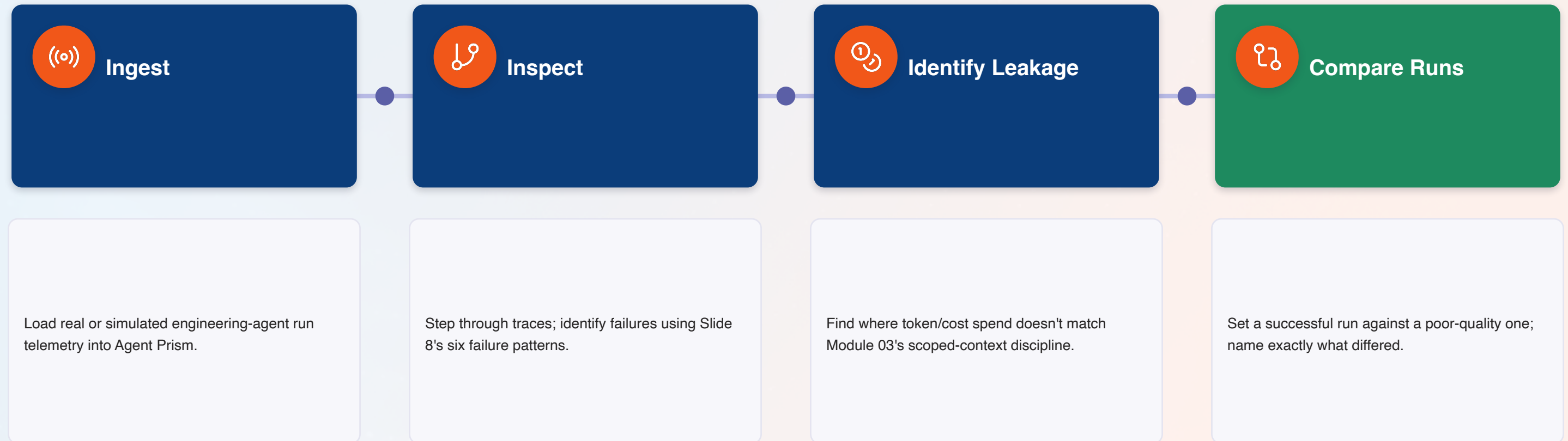
*A dashboard nobody checks is decoration — a threshold that fires is a operating rhythm.*



**Common mistake:** setting a threshold once and never revisiting it. Baselines shift as adoption grows — review the number, not just the alert.

# Hands-On Lab: Ingest, Inspect, Compare

*Four stages, real telemetry — finding out what your own engineering activity actually looks like.*



**Bring forward:** the leakage and comparison findings from today become direct inputs to Module 11's token economics and ROI model.



# Anti-Patterns: How Observability Gets Ignored

*Five habits that turn a working monitoring lens into a dashboard nobody trusts.*

## DO THIS

-  Set explicit thresholds and name who gets the alert
-  Investigate a cost trend the first time it climbs
-  Treat a failure pattern as a signal to fix the workflow
-  Track adoption by GBE, not just a single aggregate number
-  Revisit baselines as adoption grows

## NOT THAT

-  Build a dashboard and hope someone glances at it
-  Wait until the token bill is a surprise to look
-  Retry the same failing run and call it resolved
-  Report “engagement is up” without knowing where
-  Set a threshold once and never touch it again

# Module 10 Outcomes & What's Next

*Everything below should be true before we turn today's telemetry into an ROI model in Module 11.*



Can explain traces, replay & drift — and why each matters differently



Can distinguish expected token spend from genuine cost leakage



Can set a threshold and name who owns the resulting alert



Compared a successful run against a poor-quality one and named the difference



Know all six failure patterns and can identify them from real telemetry



Understand quality/control signals and adoption tracking as two distinct questions



Ingested real run data and inspected it for failures, hands-on



Know the five anti-patterns that make a dashboard nobody trusts



## NEXT — MODULE 11: ROI, TOKEN ECONOMICS AND DEPLOYMENT GOVERNANCE

1 hour · Cost per accepted output, saved effort, and governance decisions — built directly on today's traces, leakage findings & comparisons.

# Questions & Discussion

*Module 10 — Agent Prism for Monitoring, Observability and Enterprise Control*